Singly Linked List –
Get N^{TH} NODE
and Time Complexity.

The GetNth() function retrieves the data value stored in the node at a specified index from a linked list. To implement this, you typically traverse the linked list until you reach the node at the given index, then return its data. If the index is out of bounds, you should handle it accordingly (e.g., return null or an error message).

Program :

```
int getNthNode(Node **head, int node)
{
    if (*head == nullptr)
    {
        cout << "Error: List is empty." << endl;
        return -1;
    }
    if (node < 1 || node > lengthOfList(head))
    {
        cout << "Error: Invalid index." << endl;
        return -1;
    }
    else
    {
        Node *current = *head;
        int i = 1;
        while (i < node)
        {
            current = current->next;
            i++;
        }
        return current->data;
    }
}
…..
```

```
        case 18:
         {
             cout << "Nth node from beginning , where 1 is the first node(1 based
indexing):" << endl;
             int n;
             cout << "Enter the node: ";
             cin >> n;
             cout << "The " << n << "th node is from the beginning: " <<
getNthNode(&head, n) << endl;
             break;
         }
```

Program Analysis:

There are two criteria of input Linked List:

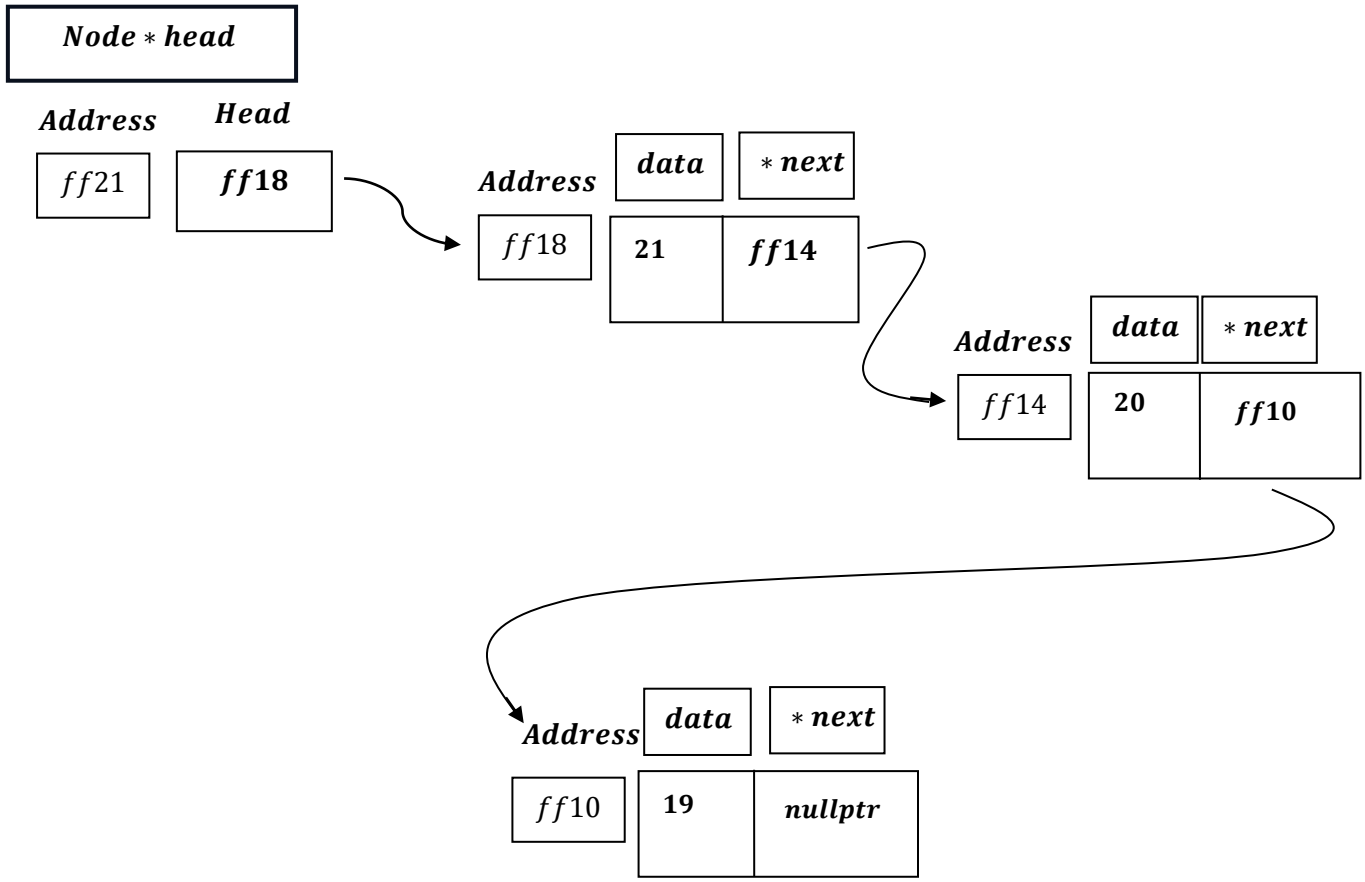
1)*Linked List is Empty*:

Pictorially depicted as:

Node * head	
Address	***Head***
$ff21$	***nullptr***

2)*Linked List is not Empty*:

Pictorially depicted as:

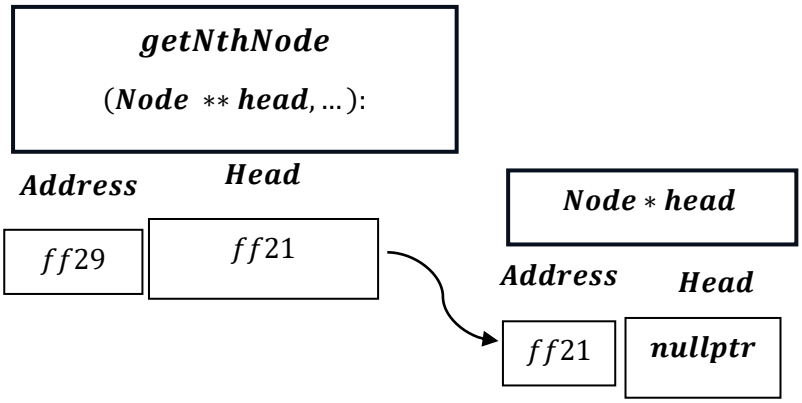


1) *When Linked List is Empty*:

```
getNthNode(&head, n);
```

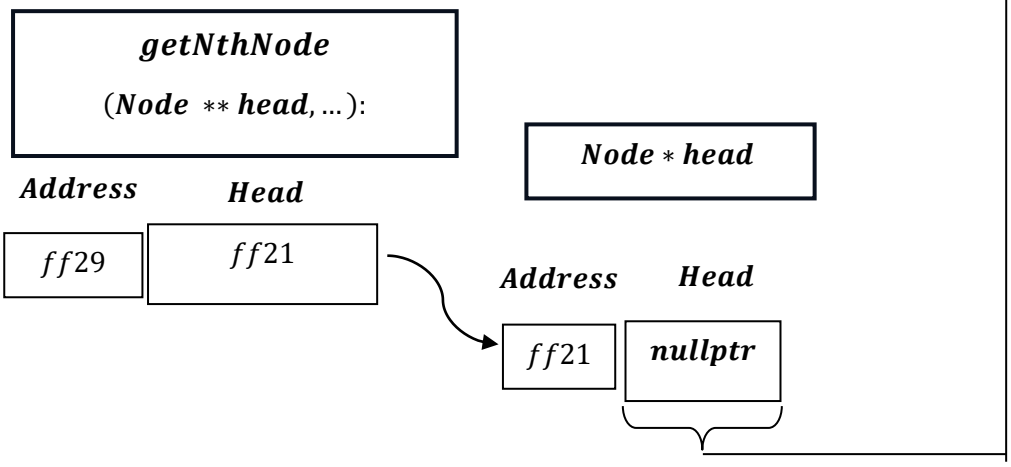
$getNthNode(\&head: ff21, 1)$

```
int getNthNode(Node **head, int node){….}
```



$if(*head == nullptr)$ *if it is true*

If true then Why true ?
*Ans: Though * head which is of Node ** head*
temporary pointer to pointer variable of the function
*, as * head is used , it dereference from Node ** head and*
*refer to Node * head global pointer to which Node ** head*
is pointing to and then its value is checked is
whether equal to nullptr or not ? This occurs during execution
and compilation of the getNthNode function.




```
    if (*head == nullptr)
    {
        cout << "Error: List is empty." << endl;
        return;
    }
```

If List is Empty , then , it prints a message : `Error: List is empty.` and returns.

Here input node index number entered by user doesn't matters.

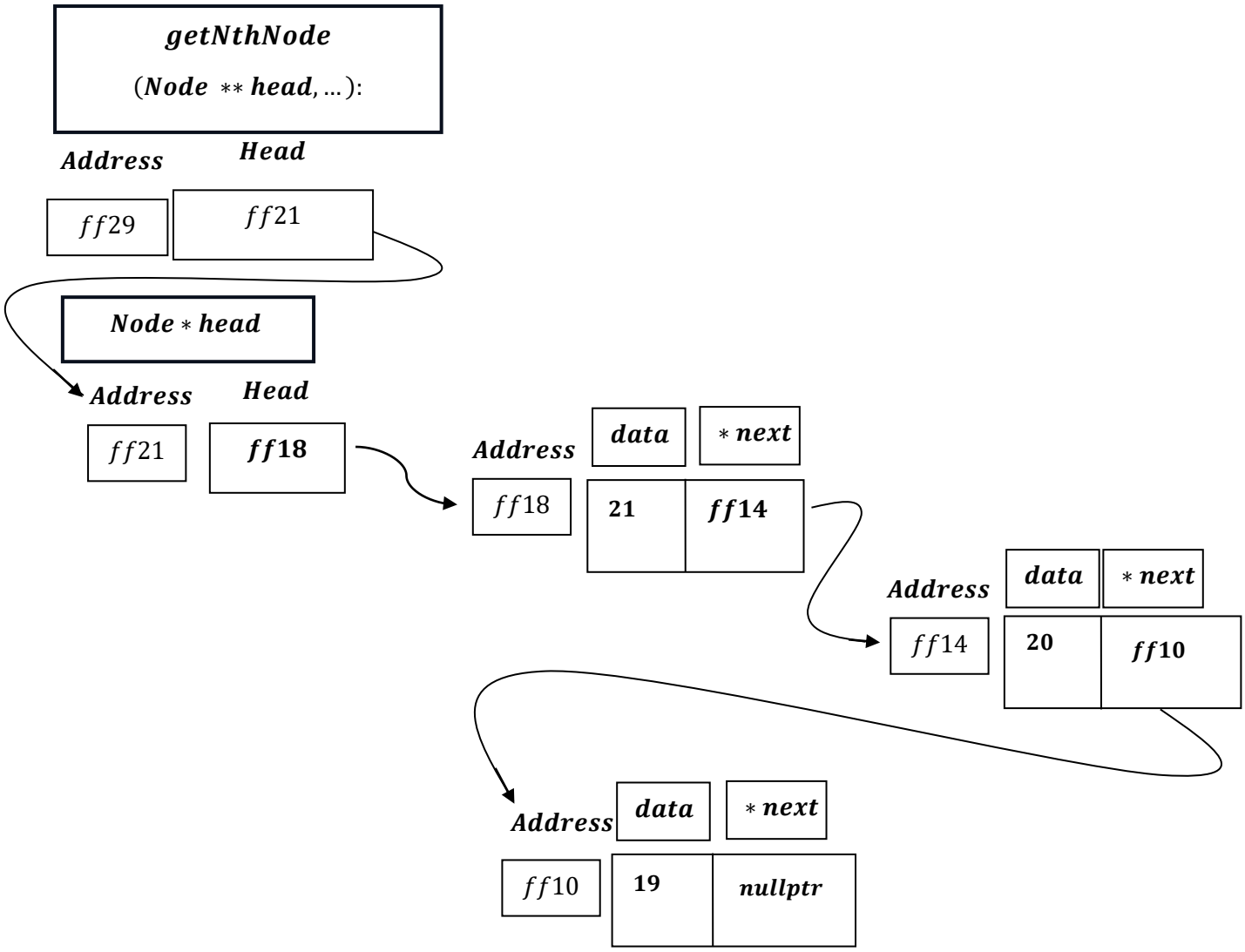
---

2) *When Node less than 1 or Node greater than Length of List*

```
getNthNode(&head, n);
```

$getNthNode(\&head: ff21, 0);$

```
int getNthNode(Node **head, int node){….}
```



```
    if (node < 1 || node > lengthOfList(head))
    {
        cout << "Error: Invalid index." << endl;
        return -1;
    }
```

$Considering\ above\ \ linked\ list, which\ have\ length: 3, i.e. Node\ 1, Node\ 2,$
$Node\ 3\ and\ there\ is\ no\ Node\ less\ than\ 1, hence\ if\ getNthNode(\&head: ff21, 0)$
$, as\ node = 0, we\ print\ a\ statement:$

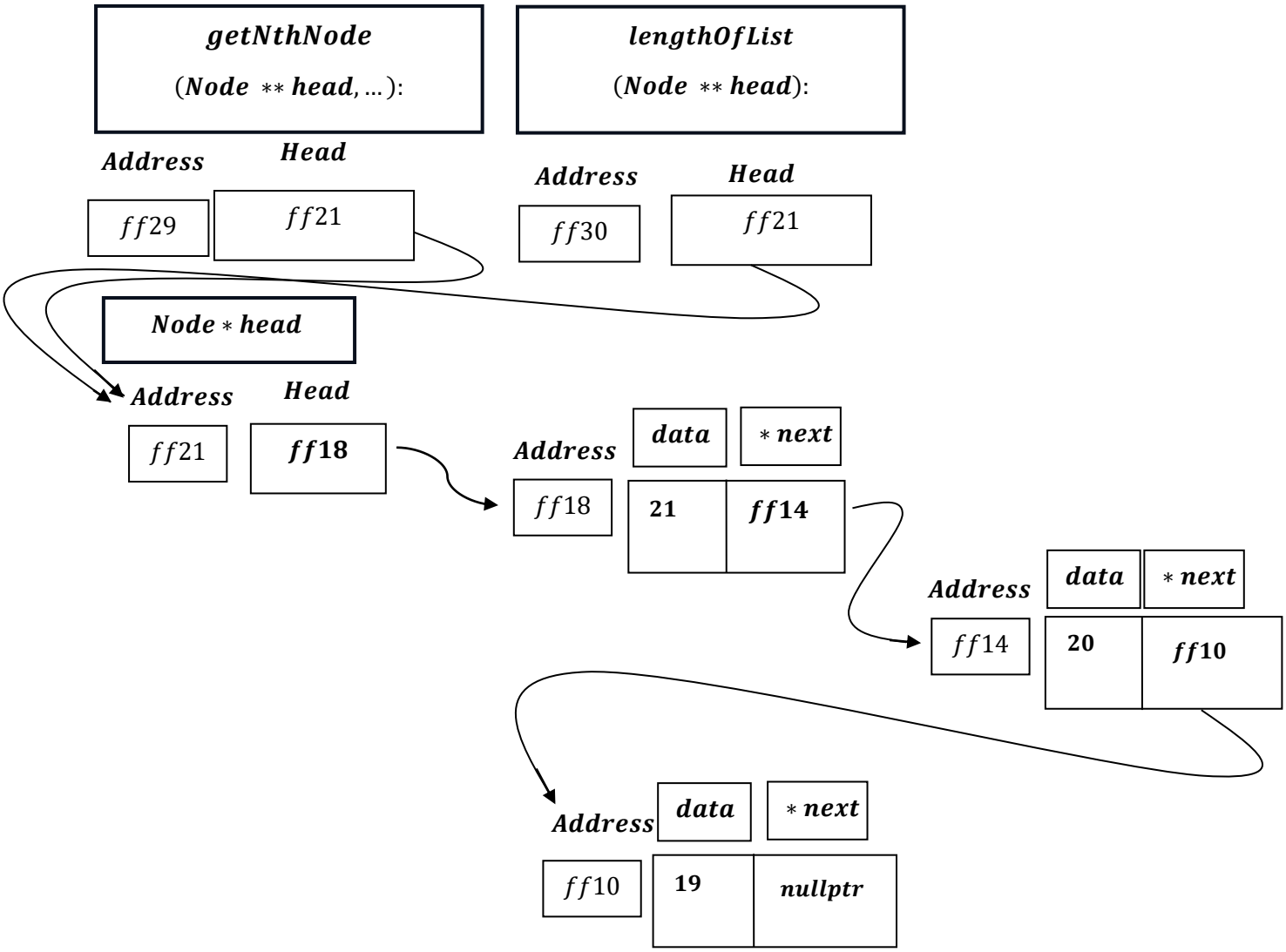
$``Error: Invalid\ index.``\ \ and\ return\ -1, as\ return\ type\ of\ the\ function\ is:$
$`int`\ [integer].$

$Also, when\ node > length(head)\ i.e. Node\ is\ greater\ than\ Length\ of\ List$
$i.e. say, 4.$

$, we\ print:$

$``Error: Invalid\ index.``\ \ and\ return\ -1, as\ return\ type\ of\ the\ function\ is:$
$`int`\ [integer].$

$\underline{Note:}$
$As\ we\ pass\ the\ value\ of\ Node\ **\ head\ in\ lengthOfList(head);$
$\rightarrow lengthOfList(ff21), and\ it\ is\ pictorially\ represented\ as:$



And then gives us the length of the list: length i.e. 3 .

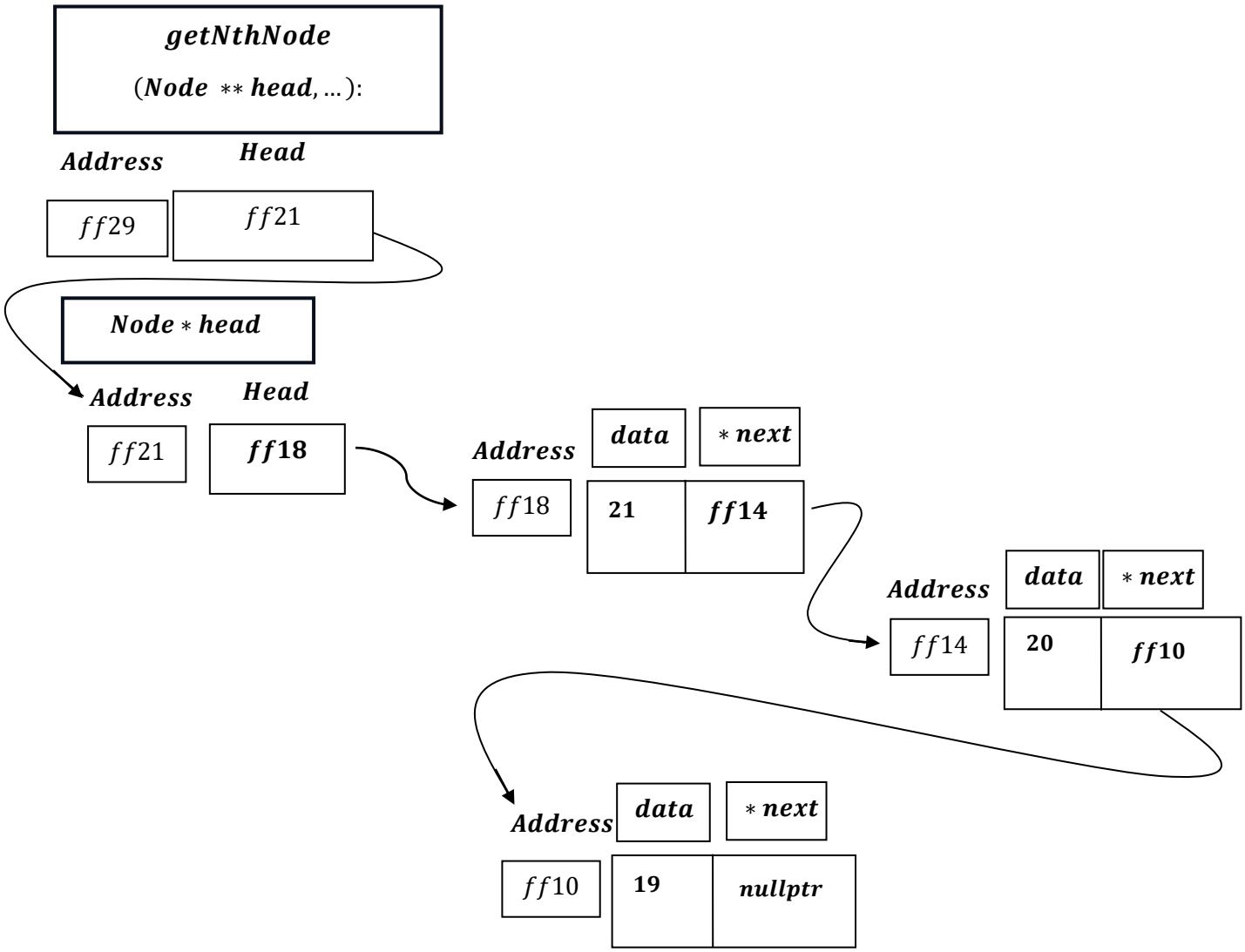
---

3) *When node is ≥ 1 and ≤ length of List*

```
getNthNode(&head, n);
```

getNthNode(&head: ff21, 3);

```
int getNthNode(Node **head, int node){….}
```



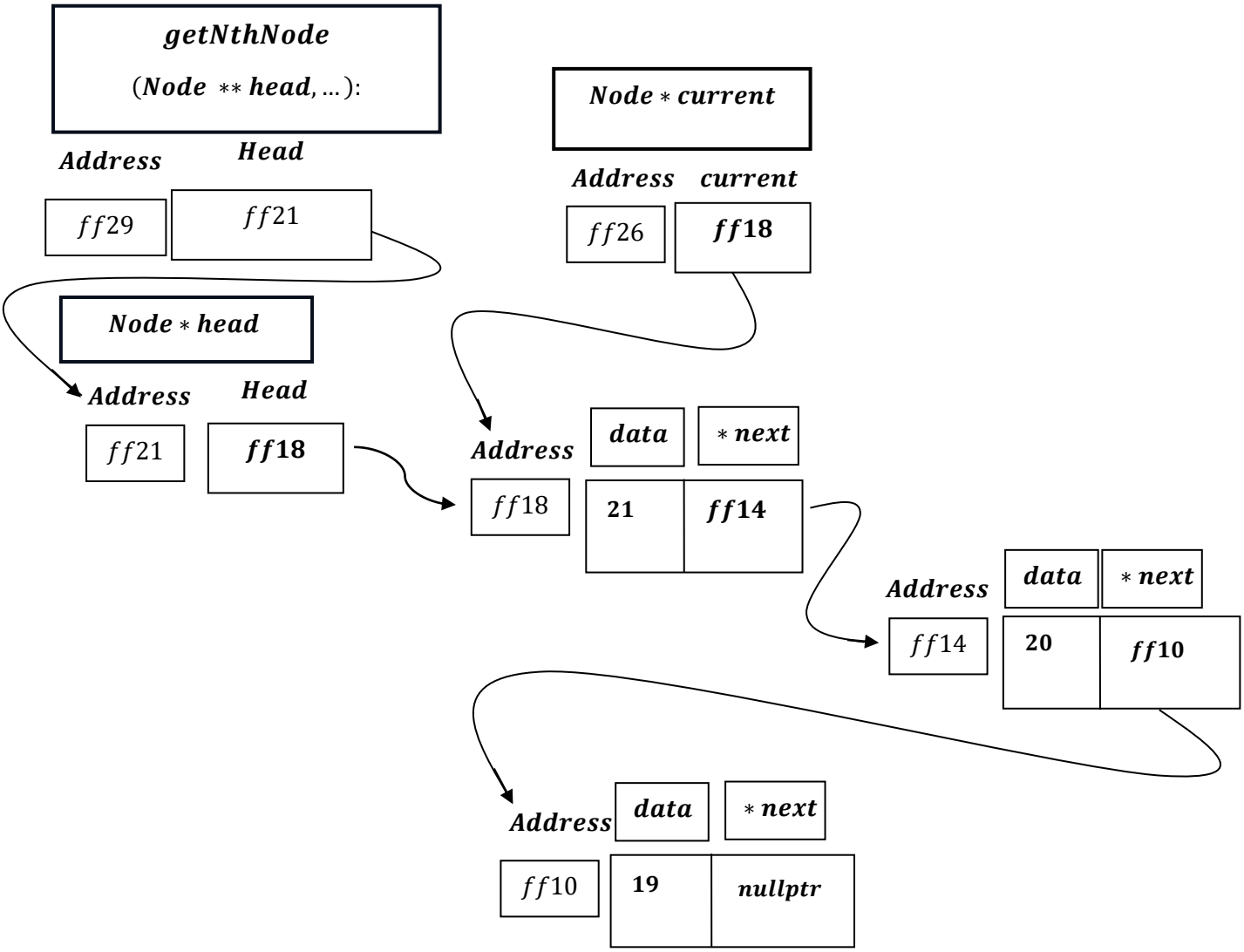

```
    else
    {
        Node *current = *head;
        int i = 1;
        while (i < node && current != nullptr)
        {
            current = current->next;
            i++;
        }
```

```
        return current->data;
    }
}
```

The current pointer assigned to value of head pointer .

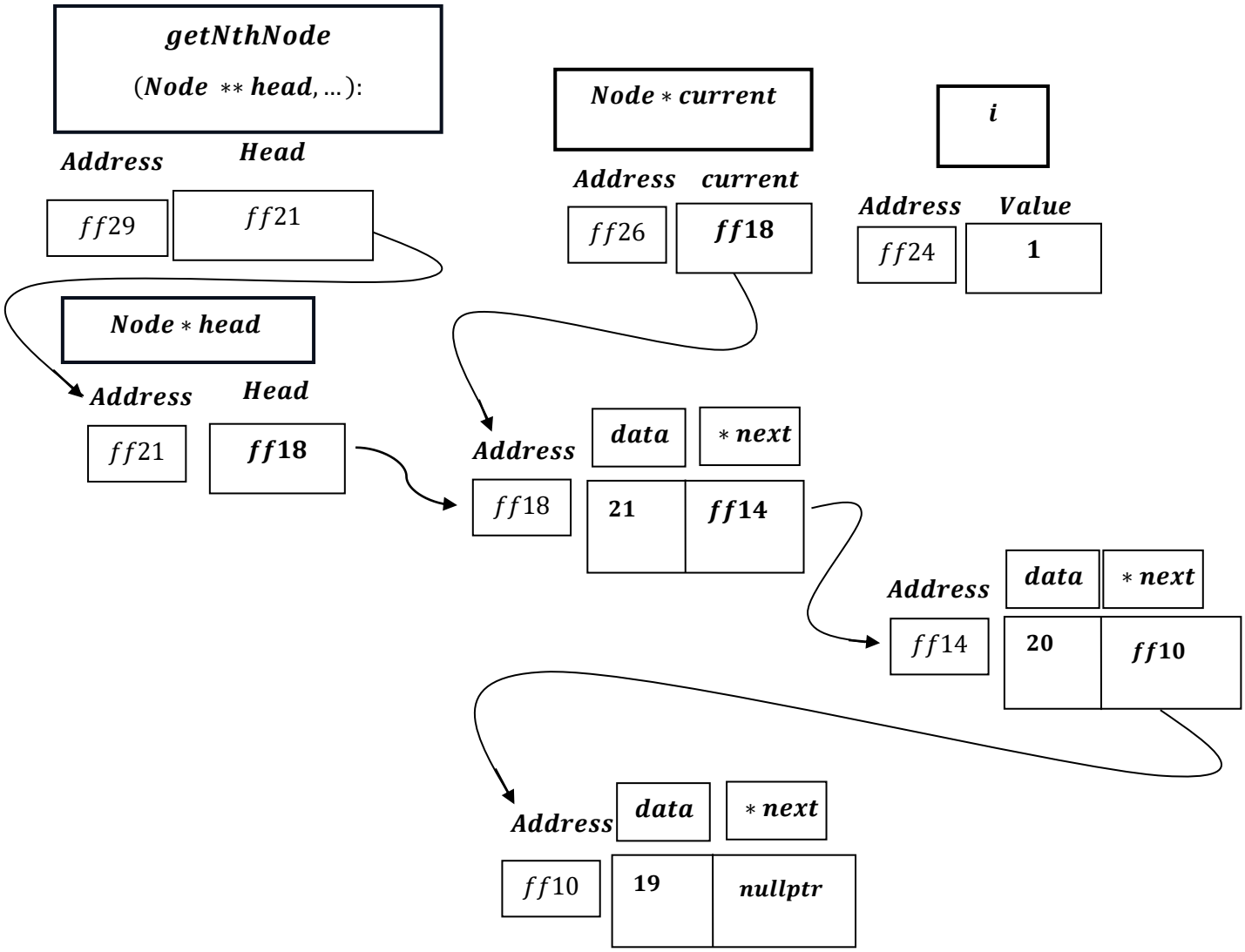
```
        Node *current = *head;
```

depicted below:



Now, i is assigned to 1;

```
        int i = 1;
```



Next,

```
        while (i < node && current != nullptr)
        {
            current = current->next;
            i++;
        }
```

Now, we notice that , current cannot be nullptr as node ≥ 1 and node < lengthOfList, therefore , as we have already made that

$node\ cannot\ be\ greater\ than\ lengthOfList, hence\ we\ omit\ the\ condition$

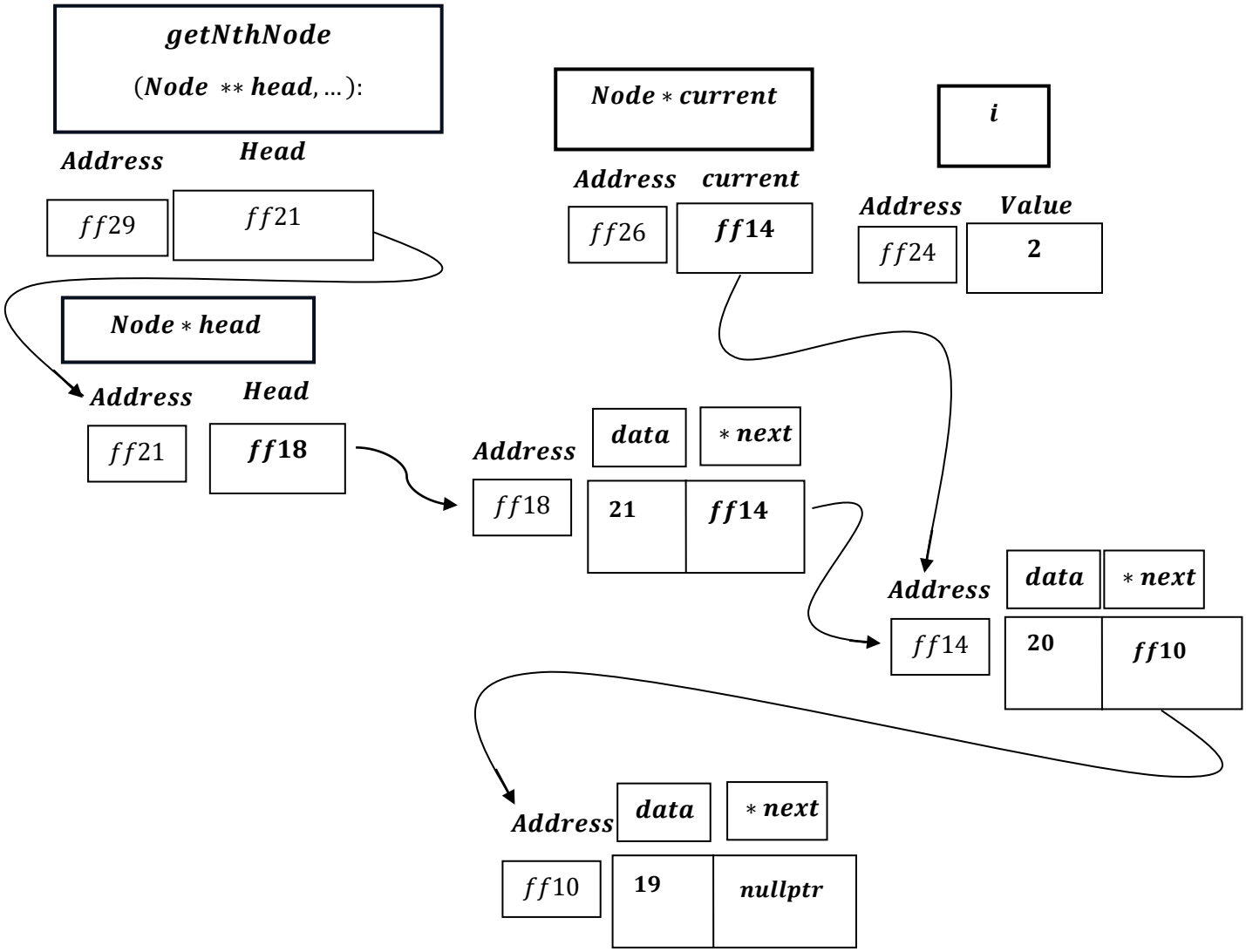
$and\ modifying\ the\ loop, we\ get$:

```
    while (i < node)
      {
          current = current->next;
          i++;
      }
```

$When\ i = 1 < node = 3[True], loop\ runs$:

$current = current \rightarrow next = ff14.$

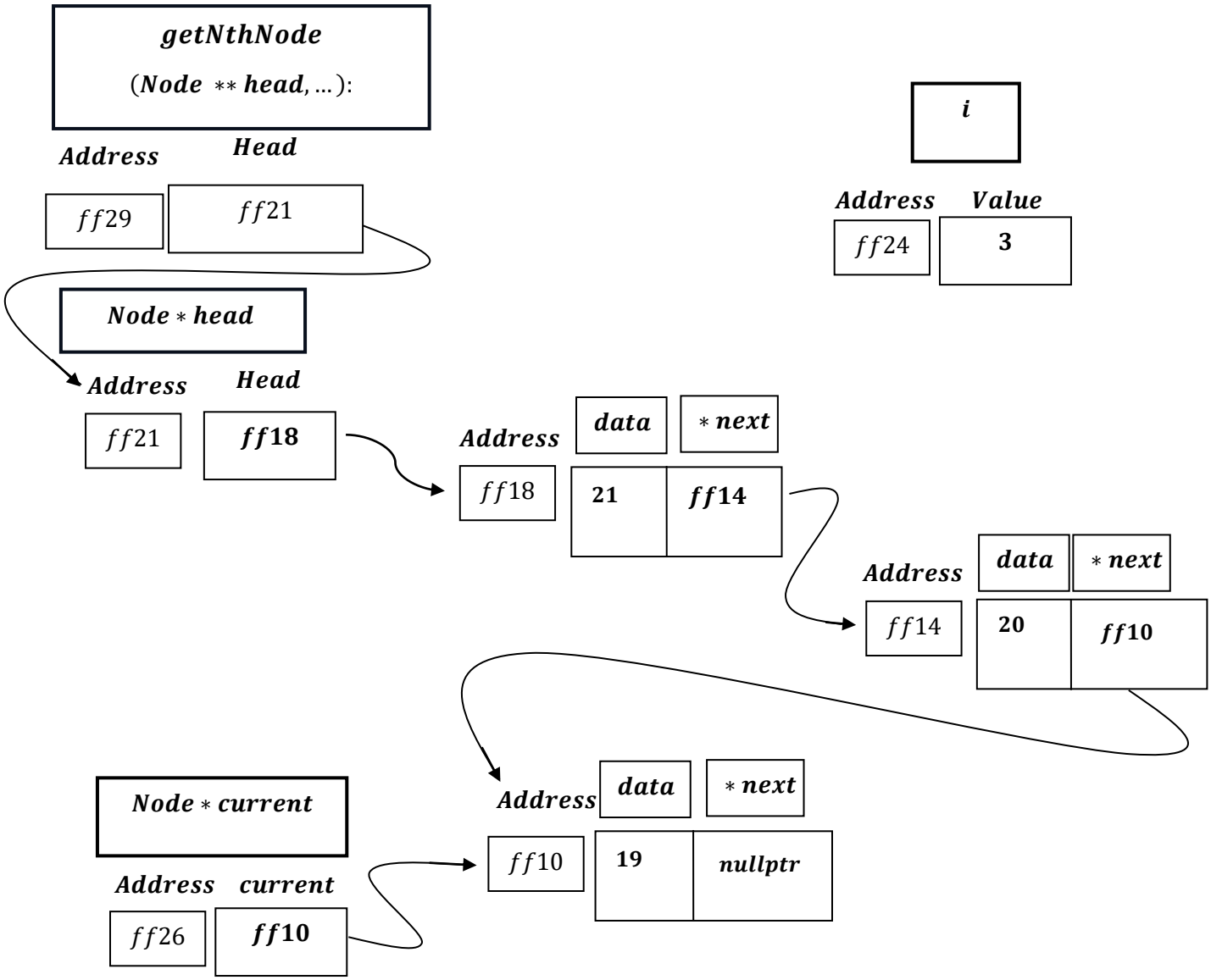
$i++ \Longrightarrow i = i + 1 = 1 + 1 = 2.$



$When\ i = 2 < node = 3[True], loop\ runs$:

$current = current \rightarrow next = ff10$.

$i++ \Longrightarrow i = i + 1 = 2 + 1 = 3$.

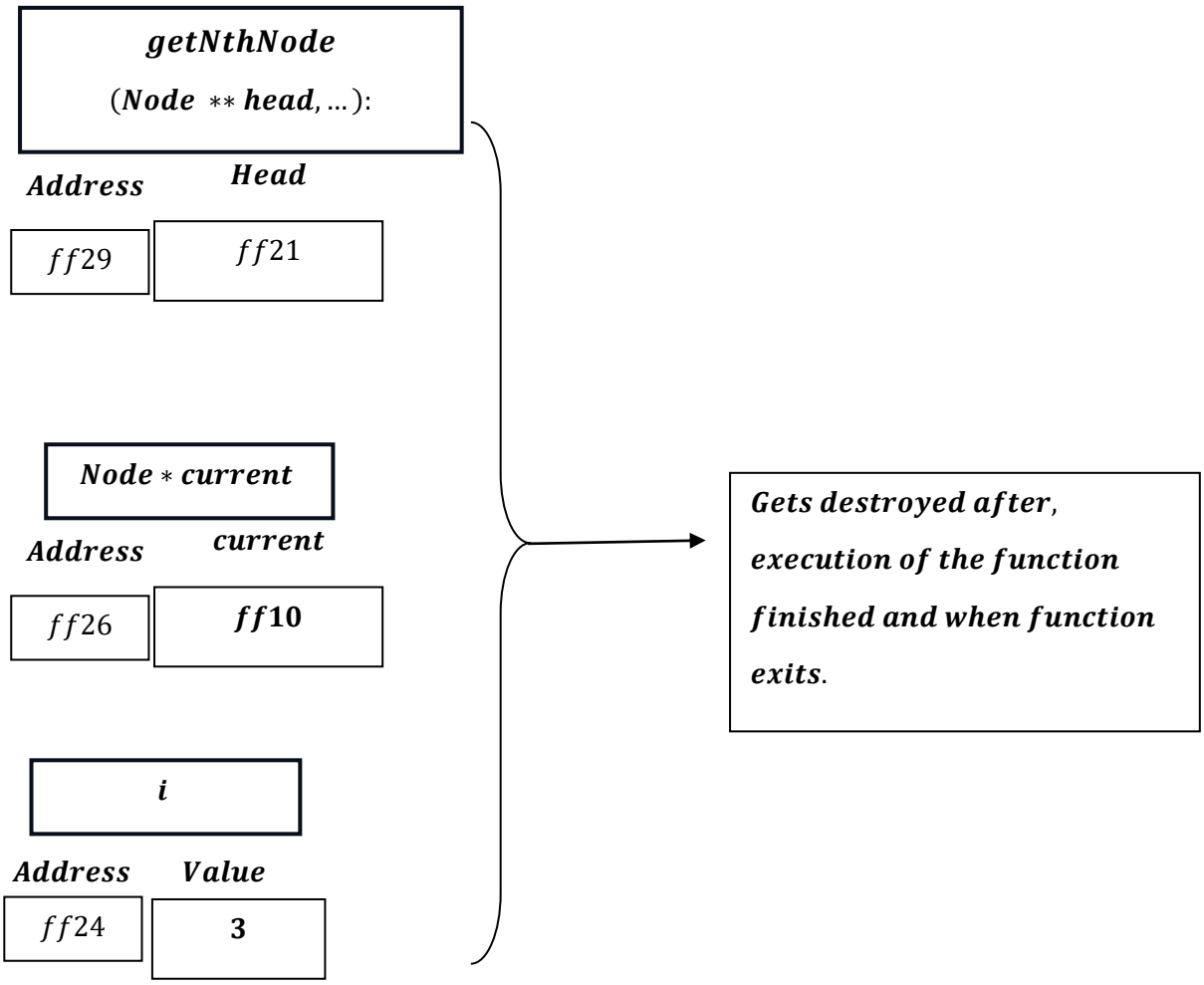


$When\ i = 3 < node = 3[False], loop\ exits.$

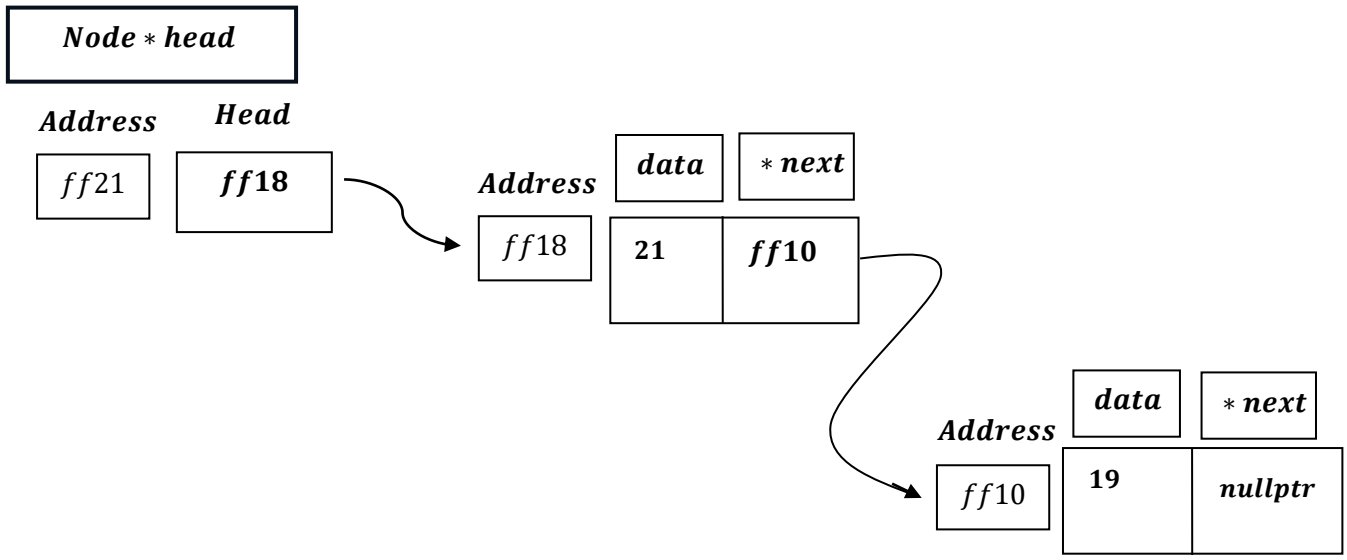
$Hence\ , return\ current \rightarrow data\ i.e. 19.$

```
        return current->data;
```

After the function module ends , all temporary pointer variables created gets destroyed, i.e.



Hence we are left with:



Now , coming to switch – case statement:

```
        case 18:
         {
             cout << "Nth node from beginning , where 1 is the first node(1 based
indexing):" << endl;
             int n;
             cout << "Enter the node: ";
             cin >> n;
             cout << "The " << n << "th node is from the beginning: " <<
getNthNode(&head, n) << endl;
             break;
         }
```

It prints: ***The 3th node is from beginning*** : **19**.

$Now, `getNthNode` step\ by\ step\ explanation:$

$1. Check\ if\ the\ List\ is\ Empty:$

$-\ The\ function\ first\ checks\ if\ the\ linked\ list\ is\ empty\ by\ checking\ if\ the$
$head\ pointer\ is\ `nullptr`.$

$-\ If\ the\ list\ is\ empty, it\ prints\ an\ error\ message\ "Error: List\ is\ empty."\ and\ returns\ `-1`.$

$2. Check\ for\ Invalid\ Index:$

$-\ The\ function\ then\ checks\ if\ the\ provided\ index\ (`node`)\ is\ valid.$

$-\ It\ checks\ two\ conditions:$

$-\ If\ `node`\ is\ less\ than\ 1, which\ means\ the\ index\ is\ invalid\ because\ list\ indices\ start\ from\ 1.$

$-\ If\ `node`\ is\ greater\ than\ the\ length\ of\ the\ list, which\ means\ the\ index\ is\ out\ of\ bounds.$

$-\ If\ either\ condition\ is\ true, it\ prints\ an\ error\ message\ "Error: Invalid\ index."\ and\ returns\ `-1`.$

$3. Traverse\ the\ List\ to\ the\ Specified\ Index:$

$-\ If\ the\ index\ is\ valid, the\ function\ proceeds\ to\ traverse\ the\ list\ to\ reach\ the\ specified\ index.$

$-\ It\ initializes\ a\ pointer\ `current`\ to\ point\ to\ the\ head\ of\ the\ list.$

$-\ It\ also\ initializes\ a\ counter\ variable\ `i`\ to\ 1, representing\ the\ current\ position\ in\ the\ list.$

$-\ The\ `while`\ loop\ runs\ until\ `i`\ is\ less\ than\ the\ specified\ index\ (`node`):$

$\quad -\ Inside\ the\ loop, it\ moves\ the\ `current`\ pointer\ to\ the\ next\ node\ (`current\ =\ current->next`).$

$\quad -\ It\ increments\ the\ counter\ `i`\ by\ 1\ (`i++`).$

$-\ When\ the\ loop\ exits, `current`\ points\ to\ the\ node\ at\ the\ specified\ index.$

4. Return the Data of the Node at the Specified Index:

– Finally, the function returns the data of the node at the specified index by accessing `current → data`.

Summary:

– The function first checks if the list is empty and returns an error if it is.
– It then checks if the provided index is valid and returns an error if it is not.
– If the index is valid, it traverses the list to reach the specified index.
– It returns the data of the node at the specified index.

This function ensures that the user gets the data of the node at the desired position in a safe and controlled manner, handling edge cases like an empty list or an invalid index.

Time Complexity of

Get N^{TH} NODE

```
int getNthNode(Node **head, int node)
{
    if (*head == nullptr)
    {
        cout << "Error: List is empty." << endl;
        return -1;
    }
    if (node < 1 || node > lengthOfList(head))
    {
        cout << "Error: Invalid index." << endl;
        return -1;
    }
    else
    {
        Node *current = *head;
        int i = 1;
        while (i < node)
        {
            current = current->next;
            i++;
        }
        return current->data;
    }
}
```

$O(1)$

$O(1)$ *or* $O(n)$

$O(1)$

$O(1)$

runs $n-1$ *time at worst case.*

$runs: 1+1+1+\cdots(n-1) = O(n).$

$runs: 1+1+1+\cdots(n-1) = O(n).$

$O(1)$

$\underline{Best\ Case}$

$\underline{1. When\ List\ is\ Empty}$:

$\rightarrow If\ the\ list\ is\ empty\ (`*head`\ is\ `nullptr`), print\ an\ error\ message\ and$
$return\ from\ the\ function. This\ takes\ O(1) time\ or\ \Omega(1) time$.

$\underline{2. When\ Node\ is\ less\ than\ 1}$

$If\ node\ less\ than\ 1\ , it\ prints: ``Error: Invalid\ index.''\ and\ then\ return\ `-1`.$
$This\ takes\ O(1) time\ or\ \Omega(1) time$.

$\underline{Worst\ Case}$

$\underline{1. When\ `node\ is\ greater\ than\ 1\ and\ greater\ than\ length\ of\ List}$

$When\ node\ is\ greater\ than\ 1\ and\ greater\ than\ length\ of\ List\ , then, the\ condition\ ,$
$is\ checked\ that\ node > lengthOfList(head). As\ we\ know\ lengthOfList(head) takes$
$`n`\ times\ i.e. O(n).$

$And\ as\ message\ gets\ print \rightarrow\ ``Error: Invalid\ Index''\ with\ return -1\ \ which\ takes\ O(1).$

$The\ function\ returns -1\ and\ exits\ in\ this\ case.$

$Therefore, total\ amount\ of\ time\ compelxity: O(n) + O(1) = O(n).$

$$Hence\ worst\ time\ complexity\ is: O(n).$$

$\underline{2. When\ `node\ is\ greater\ than\ 1\ and\ node\ entered\ is\ the\ last\ node.}$

$-\ The\ function\ first\ checks\ if\ the\ linked\ list\ is\ empty\ by\ checking\ if\ the$
$head\ pointer\ is\ `nullptr`. As\ this\ is\ not\ true\ at\ this\ time\ , the\ check\ takes\ O(1) i.e.$
$constant\ time.$

$-$ *The function then checks if the provided index (`node`) is valid. As node is greater than 1 and last node : `n` , hence this check also returns false, and takes constant time :* $O(1)$.

$-$ *Function then initializes a pointer `current` to point to the head of the list. This takes constant time :* $O(1)$.

$-$ *It also initializes a counter variable `i` to 1, representing the current position in the list. This takes constant time :* $O(1)$.

$-$ *The `while` loop runs until `i` is less than the specified index (`node`), the loop runs `n − 1` times :*

$-$ *Inside the loop, it moves the `current` pointer to the next node (`current = current → next`). It takes :* $1 + 1 + 1 + \cdots (n-1)$ *times i.e.* $n - 1$ *times.*

$-$ *It increments the counter `i` by 1 (`i++`). It takes :* $1 + 1 + 1 + \cdots (n-1)$ *times i.e.* $n - 1$ *times.*

Hence, total time complexity: $n - 1 + n - 1 = 2n - 2 = 2 \times O(n-1) = O(n) - O(1) = O(n)$.

$-$ *Finally, the function returns the data of the node at the specified index by accessing `current → data`. This takes constant time, i.e.* $O(1)$.

Hence total time complexity: $-\ O(1) + O(1) + O(1) + O(1) + O(n) + O(1) = O(n)$.

Hence worst time complexity is : $O(n)$.

Average Case

$(Type\ 1\ Based\ on\ Loop)$

$\rightarrow If\ the\ position\ is\ 1, the\ loop's\ inner\ statement\ runs\ 0\ time.$

$\rightarrow If\ the\ position\ is\ 2, the\ loop's\ inner\ statement\ runs\ 1\ time.$

$\rightarrow If\ the\ position\ is\ 3, the\ loop's\ inner\ statement\ runs\ \ 2\ time.$

.

.

$\rightarrow If\ the\ position\ is\ (N), the\ loop's\ inner\ statement\ runs\ (N-1)\ times.$

$Hence\ , \sum_{i=1}^{N-1} i = \frac{N(N-1)}{2}.$

$Now, as\ we\ take\ all\ the\ possibilites\ then\ , we\ take\ the\ best\ case\ also\ i.e.$ $O(1), when\ position = \ 1\ . Considering\ `T(P)`\ is\ time\ complexity\ of$ $position\ `P`\ .$

$$T(P) = \begin{cases} 1 & , when\ P = 1. \\ P - 1 & , when\ P = 2, 3, 4, .., N. \end{cases}$$

$Now, If\ there\ is\ `N`\ no. of\ nodes\ there\ we\ can\ find\ data\ at\ \ N\ positions.$

$For\ position\ 1, the\ probability\ of\ finding\ data\ is: \frac{1}{N}.$

$For\ position\ 2, the\ probability\ of\ finding\ data\ \ is: \frac{1}{N}.$

.

.

$$\boldsymbol{For\ position\ N, the\ probability\ of\ finding\ data\ \ is}: \frac{\mathbf{1}}{\boldsymbol{N}}.$$

$$\boldsymbol{Hence\ probability\ of\ finding\ data\ \ will\ always\ remain}: \frac{\mathbf{1}}{\boldsymbol{N}}.$$

$$\therefore\ \boldsymbol{Average\ Time\ Complexity\ \ is}:\ \boldsymbol{probability\ of\ finding\ data} \times (\boldsymbol{number\ of\ possible\ iterations} + \boldsymbol{constant\ time\ complexities\ for\ left\ over\ positions})\ .$$

$$= \left(\frac{\mathbf{1}}{\boldsymbol{N}} \times \mathbf{1} + \frac{\mathbf{1}}{\boldsymbol{N}} \times \mathbf{2} + \cdots + \frac{\mathbf{1}}{\boldsymbol{N}} \times \boldsymbol{N} - \mathbf{1}\right) + \frac{\mathbf{1}}{\boldsymbol{N}} \times \mathbf{1}$$

$$= \frac{\mathbf{1}}{\boldsymbol{N}} \times \{(\mathbf{1} + \mathbf{2} + \mathbf{3} + \cdots + \boldsymbol{N} - \mathbf{1}) + \mathbf{1}\}$$

$$= \frac{\mathbf{1}}{\boldsymbol{N}} \times \left\{\sum_{\boldsymbol{i}=\mathbf{1}}^{\boldsymbol{N}-\mathbf{1}} \boldsymbol{i} + \mathbf{1}\right\}$$

$$= \frac{\mathbf{1}}{\boldsymbol{N}} \times \left(\frac{\boldsymbol{N}(\boldsymbol{N} - \mathbf{1})}{\mathbf{2}} + \mathbf{1}\right)$$

$$= \frac{\boldsymbol{N} - \mathbf{1}}{\mathbf{2}} + \frac{\mathbf{1}}{\boldsymbol{N}}$$

$$\boldsymbol{Appplying\ Big\ Theta\ in\ the\ equation}:$$

$$= \boldsymbol{\Theta}\left(\frac{\boldsymbol{N} - \mathbf{1}}{\mathbf{2}}\right) + \boldsymbol{\Theta}\left(\frac{\mathbf{1}}{\boldsymbol{N}}\right)$$

$$\boldsymbol{Now, taking\ out\ the\ dominant\ term\ of\ numerator\ and\ denominator}:$$

$$= \Theta\left(\frac{N}{2}\right) + \Theta\left(\frac{1}{N}\right)$$

$$= \frac{1}{2} \times \Theta(N) + \Theta\left(\frac{1}{N}\right)$$

$Now\ removing\ the\ constant\ `\frac{1}{2}`\ we\ get$:

$$= \Theta(N) + \Theta\left(\frac{1}{N}\right)$$

$When\ combining\ two\ Big\ Theta\ \Theta$ **terms , largest growth rate is the result**:

$\rightarrow \Theta(N)\ grows\ linearly\ with\ `N`$.

$\rightarrow \Theta\left(\frac{1}{N}\right)\ decreases\ to\ a\ constant\ , i.e.\ when\ N\ grows\ larger\ , \frac{1}{N}\ approaches\ to\ 0.$

$This\ we\ can\ prove\ by\ limit\ and\ continuity\ also$:

$$\lim_{n\to\infty}\left(\frac{1}{n}\right) \Rightarrow \frac{1}{\infty} = 0\ \left[\frac{c}{\infty} = 0\ , where\ c\ is\ constant\right].$$

$As\ \Theta(N)'s\ growth\ rate\ is\ higher\ , therefore\ , average\ case\ is\ \Theta(N).$

$Hence, Average\ Time\ Complexity\ is:\ \Theta(N).$

$Note\ , by\ relation: \Omega \leq \Theta \leq O\ , we\ get$:

$$\Rightarrow c_1 \times g(n) \leq f(n) \leq c_2 \times g(n)$$

$$\Rightarrow c_1 \times 1 \leq \frac{N-1}{2} + \frac{1}{N} \leq c_2 \times N$$

$$\boldsymbol{Average\ Case\ Time\ Complexity[Most\ Simplest\ Alternative\ Apporach]:}$$

$$\boldsymbol{(Simplest\ approach)}$$

$\boldsymbol{For\ position\ 1, the\ probability\ of\ finding\ data\ is: \frac{1}{N}}$.

$\boldsymbol{For\ position\ 2, the\ probability\ of\ finding\ data\ is: \frac{1}{N}}$.

.

.

$\boldsymbol{For\ position\ N, the\ probability\ of\ finding\ data\ is: \frac{1}{N}}$.

$\boldsymbol{Hence\ probability\ of\ finding\ data\ always\ remain\ \frac{1}{N}, for\ position\ 1\ to\ N}$.

$$\boldsymbol{Average\ case\ complexity:\ Probability\ of\ finding\ data\ \times \begin{pmatrix} Summation \\ of\ number\ of \\ Position. \end{pmatrix}}$$

$$= \frac{1}{N} \times \left(\sum_{i=1}^{N} i \right)$$

$$= \frac{1}{N} \times \left(\frac{N(N+1)}{2} \right)$$

$$= \frac{N+1}{2}$$

$$or, \frac{N}{2} + \frac{1}{2}$$

$\boldsymbol{Appplying\ Big\ Theta\ in\ the\ equation}$:

$$= \Theta\left(\frac{N}{2}\right) + \Theta\left(\frac{1}{2}\right)$$

$$\textit{Now removing the constant `}\frac{1}{2}\textit{` we get:}$$

$$= \Theta(N) + \Theta(1)$$

$$= \Theta(N).$$

$$\textit{Hence, Average Time Complexity is}: \ \Theta\left(\frac{N+1}{2}\right) \textit{ or } \Theta(N).$$

$$\textit{Now,}$$

$$\textit{Note , by relation}: \Omega \leq \Theta \leq O \textit{ , we get:}$$

$$\Longrightarrow c_1 \times g(n) \leq f(n) \leq c_2 \times g(n)$$

$$\Longrightarrow c_1 \times 1 \leq \frac{N+1}{2} \leq c_2 \times N$$

Singly Linked List	*Cases*	*Time Complexity*
Get Nth Node	1. *Best Case*	$\Omega(1)$
	2. *Worst Case*	$O(n)$
	3. *Average Case*	$\Theta(n)$

****